Name of the module: **QAM**
Location in GIT repository: vspa_common\vspa-lib\QAM

Type of code:

- 15 VCPU assembly functions
  - 1 wrapper function to invoke modulator function
  - 1 wrapper function to invoke demodulator function
  - 6 modulator functions
  - 6 demodulator functions
  - 1 LLR packing function

Assembly Functions API:

```
extern void qamMod(
 unsigned int*              bitIn,      // ptr to the input bit streams
 vspa_complex_float16*  qamOut ,    //  ptr to the output symbols in complex HFloat
 unsigned int               N,          //  number of lines of output complex symbols
 unsigned int               M           //  Modulation order
 );

extern void qamDemod(
 vspa_complex_float16*  qamIn,       // ptr to complex input symbols in complex HFloat
 unsigned int*              llrOut,      // ptr to LLRs in 8 bit format
 float                      SNR,         // SNR parameter γ in linear scale (used to scale the LLRs)
 unsigned int               N,           // number of lines of input complex symbols
 unsigned int               M            // Modulation order
 );

extern void qamModBpsk(
 unsigned int*              bitIn,      // ptr to the input bit streams
 vspa_complex_float16*  qamOut ,    //  ptr to the output symbols in complex HFloat
 unsigned int               N           //  number of lines of output complex symbols
 );

extern void qamModQpsk(
 unsigned int*              bitIn,      // ptr to the input bit streams
 vspa_complex_float16*  qamOut ,    //  ptr to the output symbols in complex HFloat
 unsigned int               N           //  number of lines of output complex symbols
 );

extern void qamMod16(
 unsigned int*              bitIn,      // ptr to the input bit streams
 vspa_complex_float16*  qamOut ,    //  ptr to the output symbols in complex HFloat
 unsigned int               N           //  number of lines of output complex symbols
```

```
  );

extern void qamMod64(
  unsigned int*              bitIn,        // ptr to the input bit streams
  vspa_complex_float16*  qamOut ,     //  ptr to the output symbols in complex HFloat
  unsigned int               N             //  number of lines of output complex symbols
  );

extern void qamMod256(
  unsigned int*              bitIn,        // ptr to the input bit streams
  vspa_complex_float16*  qamOut ,     //  ptr to the output symbols in complex HFloat
  unsigned int               N             //  number of lines of output complex symbols
  );

extern void qamMod1024(
  unsigned int*              bitIn,        // ptr to the input bit streams
  vspa_complex_float16*  qamOut ,     //  ptr to the output symbols in complex HFloat
  unsigned int               N             //  number of lines of output complex symbols
  );


extern void qamDemodBpsk(
  vspa_complex_float16*  qamIn,        // ptr to complex input symbols in complex HFloat
  unsigned int*              llrOut,       // ptr to LLRs in 8 bit format
  float                      SNR,          // SNR parameter γ in linear scale (used to scale the LLRs)
  unsigned int               N,            // number of lines of input complex symbols
  );

extern void qamDemodQpsk(
  vspa_complex_float16*  qamIn,        // ptr to complex input symbols in complex HFloat
  unsigned int*              llrOut,       // ptr to LLRs in 8 bit format
  float                      SNR,          // SNR parameter γ in linear scale (used to scale the LLRs)
  unsigned int               N,            // number of lines of input complex symbols
  );

extern void qamDemod16(
  vspa_complex_float16*  qamIn,        // ptr to complex input symbols in complex HFloat
  unsigned int*              llrOut,       // ptr to LLRs in 8 bit format
  float                      SNR,          // SNR parameter γ in linear scale (used to scale the LLRs)
  unsigned int               N,            // number of lines of input complex symbols
  );

extern void qamDemod64(
  vspa_complex_float16*  qamIn,        // ptr to complex input symbols in complex HFloat
  unsigned int*              llrOut,       // ptr to LLRs in 8 bit format
```

```
 float                  SNR,        // SNR parameter γ in linear scale (used to scale the LLRs)
 unsigned int           N,          // number of lines of input complex symbols
 );

extern void qamDemod256(
 vspa_complex_float16*  qamIn,      // ptr to complex input symbols in complex HFloat
 unsigned int*          llrOut,     // ptr to LLRs in 8 bit format
 float                  SNR,        // SNR parameter γ in linear scale (used to scale the LLRs)
 unsigned int           N,          // number of lines of input complex symbols
 );

extern void qamDemod1024(
 vspa_complex_float16*  qamIn,      // ptr to complex input symbols in complex HFloat
 unsigned int*          llrOut,     // ptr to LLRs in 8 bit format
 float                  SNR,        // SNR parameter γ in linear scale (used to scale the LLRs)
 unsigned int           N,          // number of lines of input complex symbols
 );

extern void qamLlrPack(
 __fp16*                llrIn,      // ptr to real input LLRs in HFloat (intermediate value of LLRs)
 unsigned int*          llrOut,     // ptr to output LLRs in 8-bit format
 float                  SNR,        // SNR parameter γ in linear scale (used to scale the LLRs)
 unsigned int           N,          // number of lines of output LLRs
 );
```

Implementation plan

This module supports modulation and demodulation of various modulation orders, # defined in the header file:

- # define QAM_BPSK 1
- # define QAM_QPSK 2
- # define QAM_16   4
- # define QAM_64   6
- # define QAM_128   8
- # define QAM_1024 10

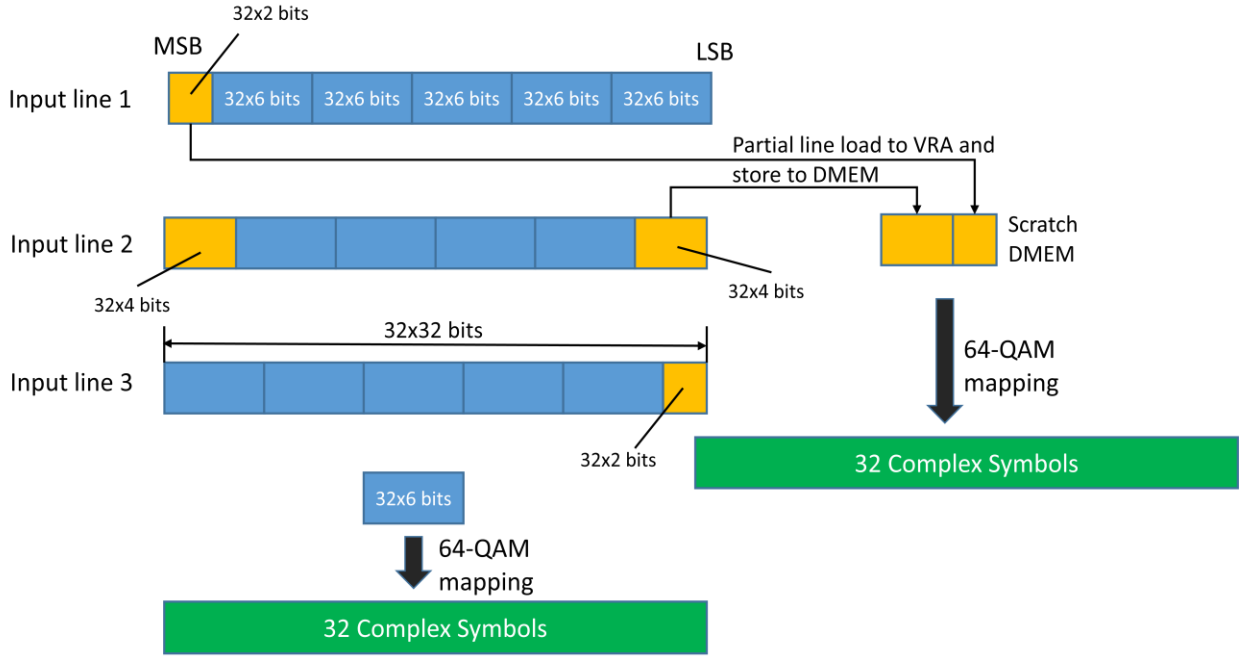
The constellation mapping is based on the WiFi standard (IEEE Std 802.11-2012). The C invoking function qamMod will call one of the modulator assembly functions, based on input parameter M, which will be one of the values defined above. The demodulator function will work in the same manner.

Modulation (qamMod)

VSPA has the capability to convert the bit stream from memRead bus to QAM complex HFixed symbols and load to register files, using the "ld.qam Rx" instruction. The HFixed symbol needs to be scaled with a normalization factor corresponding to the specific QAM scheme. This scaling constant will be # defined in the header file.

The last line of output may be a partial line. For example, if we have 80 output symbols (two and a half lines), the lower half of last output line will be valid data while the upper half will be any random data which we can just ignore.

The modulation for 64-QAM and 1024-QAM is more complicated than the other schemes, since one line of input bits will generate QAM symbols of several full lines and a partial line. See the figure below illustrating the mapping for 64-QAM.



After 5 segments of input bits (each segment has 32x6 bits: equivalent to map to a full output line of 32 QAM symbols), there is a residual of 32x2 bits. We need to move and combine this piece of data with the first 32x4 bits of the next line to form a new segment and modulate to output a full line of QAM symbols. This can be achieved by partial line processing (loading to VRA and storing to DMEM) feature of VSPA 2. It is obvious that 3 lines of input bits will output 16 lines of QAM symbols. Consequently the 64-QAM modulator function must have a main loop which processes 3 input lines and outputs 16 lines for each iteration. If the total number of lines of output symbols is N, we denote $a = floor\left(\frac{N}{16}\right); b = mod(N, 16)$. The 64-QAM modulator will first process a iterations of main loop. The remaining b lines of output will be processed next (without going to

the whole loop). Note a can be zero in which case there are less than 16 lines of outputs and the main loop should be bypassed.

The processing of 1024-QAM can be achieved in a similar manner. In this case each iteration will process 5 input lines and output 16 lines of QAM symbols.

Demodulation (qamDemod)

Demodulation is implemented with the qamDemod function. Denote the complex noisy input signal as $y = y_I + j * y_Q$, we will demodulate the signal for various QAM schemes and calculate the LLRs as follows:

BPSK:

$$D_0 = y_I$$

QPSK:

$$D_0 = y_I \qquad D_1 = y_Q$$

16-QAM:

$$D_0 = y_I \qquad D_2 = y_Q$$
$$D_1 = -|D_0| + K \qquad D_3 = -|D_2| + K$$

64-QAM:

$$D_0 = y_I \qquad D_3 = y_Q$$
$$D_1 = -|D_0| + 2K \qquad D_4 = -|D_3| + 2K$$
$$D_2 = -|D_1| + K \qquad D_5 = -|D_4| + K$$

256-QAM:

$$D_0 = y_I \qquad D_4 = y_Q$$
$$D_1 = -|D_0| + 4K \qquad D_5 = -|D_4| + 4K$$
$$D_2 = -|D_1| + 2K \qquad D_6 = -|D_5| + 2K$$
$$D_3 = -|D_2| + K \qquad D_7 = -|D_6| + K$$

1024-QAM:

$$D_0 = y_I \qquad D_5 = y_Q$$
$$D_1 = -|D_0| + 8K \qquad D_6 = -|D_5| + 8K$$
$$D_2 = -|D_1| + 4K \qquad D_7 = -|D_6| + 4K$$
$$D_3 = -|D_2| + 2K \qquad D_8 = -|D_7| + 2K$$
$$D_4 = -|D_3| + K \qquad D_9 = -|D_8| + K$$

where K is twice the normalization factor (# defined in the header file). Note the outputs of this function are intermediate values of LLRs, which need to be scaled by SNR value to obtain the final LLR values. The scaling of SNR will be implemented in the qamLlrPack function.

We can implement the above equations efficiently using the VSPA AU operation in a vectorized manner. The complex input signal will be processed and the output will be real HFloat data and store in a local scratch buffer labeled _qamScratch. The following shows how the outputs of this function are stored in the scratch buffer for 64-QAM. The extension to other modulation schemes is straightforward.

```
MSB                                                          LSB
S32(D3) S32(D0) ..... S2(D3)   S2(D0)   S1(D3)   S1(D0)
S32(D4) S32(D1) ..... S2(D4)   S2(D1)   S1(D4)   S1(D1)
S32(D5) S32(D2) ..... S2(D5)   S2(D2)   S1(D5)   S1(D2)
S64(D3) S64(D0) ..... S34(D3)  S34(D0)  S33(D3)  S33(D0)
S64(D4) S64(D1) ..... S34(D4)  S34(D1)  S33(D4)  S33(D1)
S64(D5) S64(D2) ..... S34(D5)  S34(D2)  S33(D5)  S33(D2)
                      .....    .....
```

where $S_i(D_j)$ represents the j-th bit of i-th symbol.

After we calculated the intermediate LLRs in HFloat, qamLlrPack function will be called inside the qamDemod assembly function (e.g. qamDemod16). The real HFloat data in scratch buffer will be the input to this llrPack function. Denote the input to this function D_i , we need to scale the input LLRs by SNR parameter γ (assuming identical SNR for all subcarriers/symbols). Before this vector-multiply-scalar operation, we set a saturation threshold θ, and multiply the parameter γ with the reciprocal of θ, and get $\gamma' = \gamma/\theta$. Usually, θ is chosen to be an integer power of 2, and this operation can be easily implemented by "floatx2n" operation of gX register. The input LLRs D_i are then multiplied with γ' and the results are stored to VRA in HFixed precision, which is in the range of -1 and +1 ($|\gamma' D_i| = |\gamma D_i/\theta| < 1$). Hence the final LLRs (i. e. γD_i) are bounded by $-\theta$ and $+\theta$: any LLRs beyond that threshold will be clipped to $-\theta$ and $+\theta$. Finally LLRs will be quantized to 256 levels (8-bit llr) or 16 levels (4-bit levels) with the VSPA "st.llr" instruction and packed and stored to DMEM as the final LLR output.

The initial version of this function will implement a hard coded $\theta = 16$.

DMEM requirements:

Variable	Minimum size (half-words)	Minimum alignment (half-words)	Allocated by
bitIn	2M	64 (vector aligned)	Caller
qamOut	64	64 (vector aligned)	Caller
qamIn	64	64 (vector aligned)	Caller
llrOut	16M	64 (vector aligned)	Caller
qamScratch	32M	64 (vector aligned)	Function

where M is the modulation order defined in the beginning of implementation plan section.

Test plan:

- **Feature that are going to be tested:**
  - Different K value
  - Different N value (number of lines of complex symbols)
  - Different modulation order M